Embeddings

Lesson 2: From Tokens to Document Embeddings

Agenda

- **Heuristic pooling** — BOW, TF-IDF, mean-pooling
- **Recurrent encoders** — the RNN, and how it respects order
- **The vanishing-gradient problem** — why plain RNNs forget the distant past
- **The gated cell: LSTM** — how gates and an additive path fix it (with a remark on ELMo)
- **Attention** — a first look, as *weighted pooling*

Learning Objectives

By the end of this lesson, you'll be able to:

- Build BOW, TF-IDF, and mean-pooled document embeddings by hand, and explain why all three ignore word order.
- Follow the RNN recurrence step by step on tiny numbers, and see how it encodes word order.
- See why gradients vanish (or explode) through time in a plain RNN, and read the decay off a numeric table.
- Walk through the LSTM's gate equations gate by gate, and explain how its additive cell path defeats vanishing gradients.
- Describe attention as a weighted-pooling operation and compute a worked 3-token example.

Why this matters

Lesson 1 gave every *token* a vector. But most tasks — classification, search, retrieval — need **one vector for a whole document**: a sentence, an article, a query. How you combine token vectors into one document vector is a genuinely open design question, and the choice has real consequences.

Part I — Heuristic Pooling

From tokens to one document vector

We now have a vector per **token** (lesson 1). A sentence or document is many tokens — so how do we get **one** vector representing the whole thing?

The cheapest strategy: throw the token vectors in a bag and **pool** them — summarize, don't model. Three flavors, from crudest to most useful:

1. **BOW** (bag of words) — count tokens.
2. **TF-IDF** — count tokens, but down-weight the boring ones.
3. **Mean-pooling** — average the *learned* embeddings.

All three share a blessing (simple, fast, strong baselines) and a curse: `dog bites man` = `man bites dog`.

Bag of Words (BOW), step by step

BOW represents a document as a length-$|V|$ vector of token **counts**, built on the one-hot encodings from lesson 1. The recipe, worked on d_1 = `this is a lovely paper lantern that is also handmade`:

1. **Construct the vocabulary**: all distinct tokens of the document → $|V| = 9$.
2. **Count occurrences** of each vocabulary token in the document.
3. **Build the document vector**: stack the counts.

$$x_{\text{BOW}} = \sum_{t=1}^{T} x_{\text{OHE}}(w_t) \in \mathbb{R}^{|V|} \tag{1}$$

Each token adds a 1 to its own slot — BOW is literally the **sum of one-hots**. **Dimension check:** each $x_{\text{OHE}} \in \{0,1\}^{|V|}$, so the sum is $|V|$-dimensional ✓.

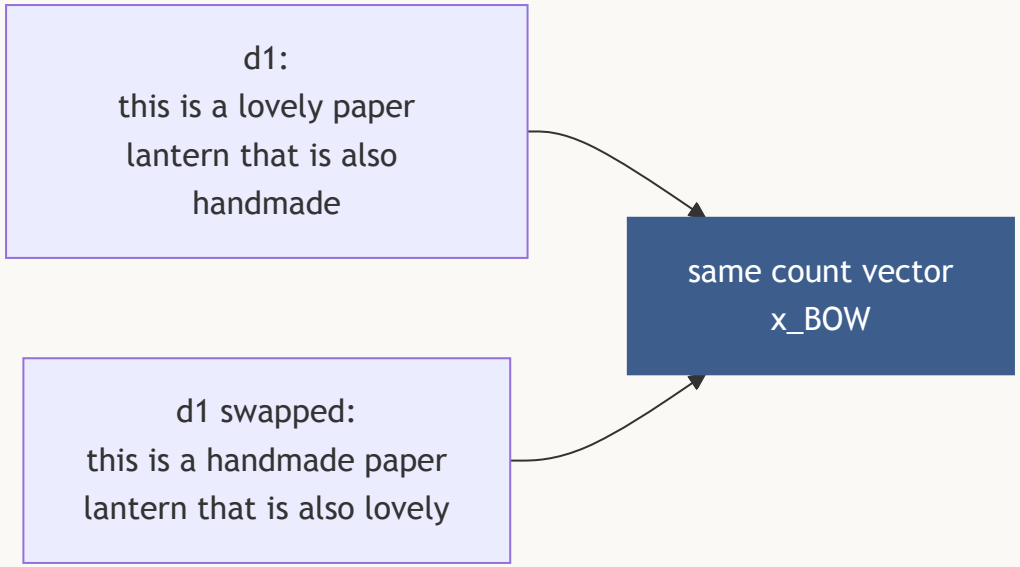
BOW: the counts

a	also	handmade	is	lantern	lovely	paper	that	this
1	1	1	2	1	1	1	1	1

d_1 = `this is a lovely paper lantern that is also handmade` — 10 tokens, 9 vocabulary entries (`is` repeats).

$x_{\mathrm{BOW}}(d_1)$ stacks these nine counts into a 9-dimensional vector.

BOW's defining limitation: order is discarded

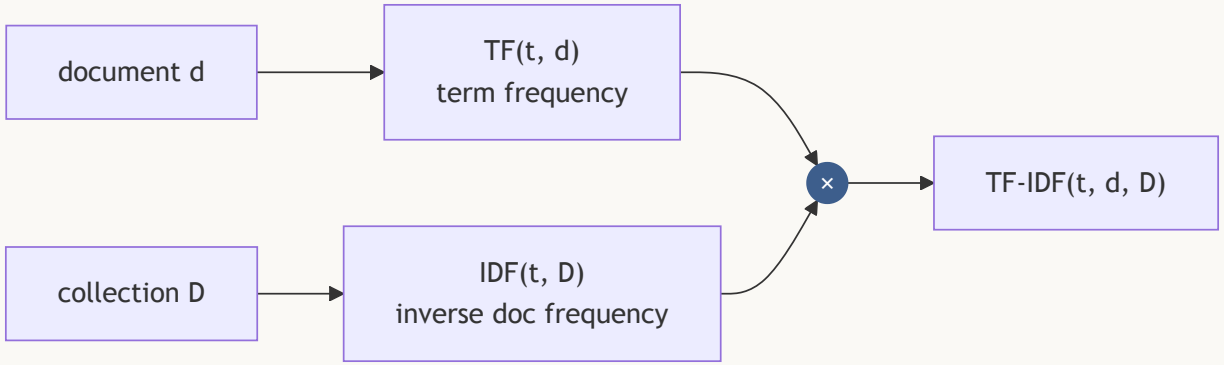


Swap `lovely` and `handmade` — d_1' = `this is a handmade paper lantern that is also lovely` — and the counts are **identical**: $x_{\mathrm{BOW}}(d_1) = x_{\mathrm{BOW}}(d_1')$.

Two sentences with different meanings (here mildly, in general wildly), **one vector**.

TF-IDF: down-weight the boring words

TF-IDF re-weights the raw counts so words that are common **everywhere** stop dominating — it "accounts for both the frequency of a word in a given document and its prevalence across all documents", filtering out words like `the` to focus on the words that actually identify a document.



Term frequency (TF): how often, here

The first factor asks the easy question: **how often does this word show up, in this document?** Normalize by document length so a long document and a short one are comparable.

$$\mathrm{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \tag{2}$$

On d_1 = `this is a lovely paper lantern that is also handmade` (10 tokens) — same counts as the BOW table, now normalized by length:

a	also	handmade	is	lantern	lovely	paper	that	this
0.10	0.10	0.10	0.20	0.10	0.10	0.10	0.10	0.10

⚠️ TF alone is fooled: `is` scores **higher** than `lantern`, even though `lantern` is the word that actually tells you what the document is about. TF can't tell common from topical — it only counts.

Inverse document frequency (IDF): how rare, overall

The second factor asks the opposite question: **how rare is this word across the *whole* collection?** A word every document uses tells you nothing about which document you're reading.

$$\mathrm{IDF}(t, \mathcal{D}) = \ln \frac{N_{\mathcal{D}}}{N_{t,\mathcal{D}}} \tag{3}$$

On a 3-document collection ($N_{\mathcal{D}} = 3$): d_1 (above), d_2 = `a paper crane is folded`, d_3 = `today is a great day`.

- `is`, `a` appear in **all 3** docs → $\mathrm{IDF} = \ln(3/3) = 0$ — common, so uninformative.
- `paper` appears in **2** → $\ln(3/2) \approx 0.405$.
- `lantern` appears in only **1** → $\ln(3/1) \approx 1.099$ — rare, so informative.

TF-IDF: the product

Multiply the two factors. The product is large **only** when a word is frequent *here* **and** rare *overall* — exactly the informative words. Stack one TF-IDF value per vocabulary term and the output is a **document vector** $x_{\text{TF-IDF}}(d) \in \mathbb{R}^{|V|}$ — same shape as BOW's eq. (1), just re-weighted.

$$\text{TF-IDF}(t, d, \mathcal{D}) = \text{TF}(t, d) \times \text{IDF}(t, \mathcal{D}) \tag{4}$$

term t	$\text{TF}(t, d_1)$	$\text{IDF}(t, \mathcal{D})$	$\text{TF-IDF}(t, d_1)$
`is`	0.20	0	**0**
`paper`	0.10	0.405	0.041
`lantern`	0.10	1.099	**0.110**

The document's **most frequent** word (`is` , TF $= 0.20$) scores **exactly zero** — it identifies nothing. The topical word `lantern` tops the ranking. That inversion is the entire point of IDF.

Mean-pooling: average the *learned* vectors

Instead of counting tokens, average their **dense** embeddings from lesson 1:

$$x_{\text{doc}} = \frac{1}{T} \sum_{t=1}^{T} v_{w_t} \in \mathbb{R}^d \tag{5}$$

Micro-example (2-D toy vectors — keep these numbers, we reuse them next section):
$v_{\texttt{paper}} = (1, 0), v_{\texttt{lantern}} = (0, 1) \rightarrow x_{\text{doc}} = (0.5,\ 0.5)$ — and *the same* $(0.5, 0.5)$ if the tokens arrive in the **opposite order**.

Mean-pooling: why it's a strong baseline

- **Dense**, not sparse: each v_{w_t} in eq. (5) **is** word2vec/GloVe's own output vector for that token ($d \sim 100$, not $|V|$) — so averaging them inherits word2vec's semantics, and two documents about lanterns land near each other even with **no shared words** because of semantics.
- On mini-lab D's DBpedia-14 retrieval task (14 classes), mean-pooling reaches **≈0.91 precision@10** against a 0.071 chance floor — before any sequence model enters the picture.
- A cheap upgrade: **you can combine both halves of this section** — instead of the plain mean, weight each token's word2vec vector by its TF-IDF before averaging.

Heuristic pooling: strengths & limitations

	BOW / TF-IDF	mean-pooled embeddings
dimension	$\|V\|$ (10^4–10^5), sparse	d ($\sim$ 50–1000), dense
"similar" means	share **literal words**	share **contexts**
lantern-doc vs. lamp-doc, no shared words	similarity 0	close — the token vectors already know
word order	none	none
training needed	none (counting)	a pretrained E (word2vec/GloVe)

- ✅ Simple, fast, transparent, no training required (BOW/TF-IDF) or reuse of an existing E (mean-pool).
- ✅ Stubbornly strong baselines — beating them requires a model that actually *uses* order.
- ⚠️ **All three are order-blind.** Fixing that is the rest of this lesson.

Part II — Recurrent Encoders

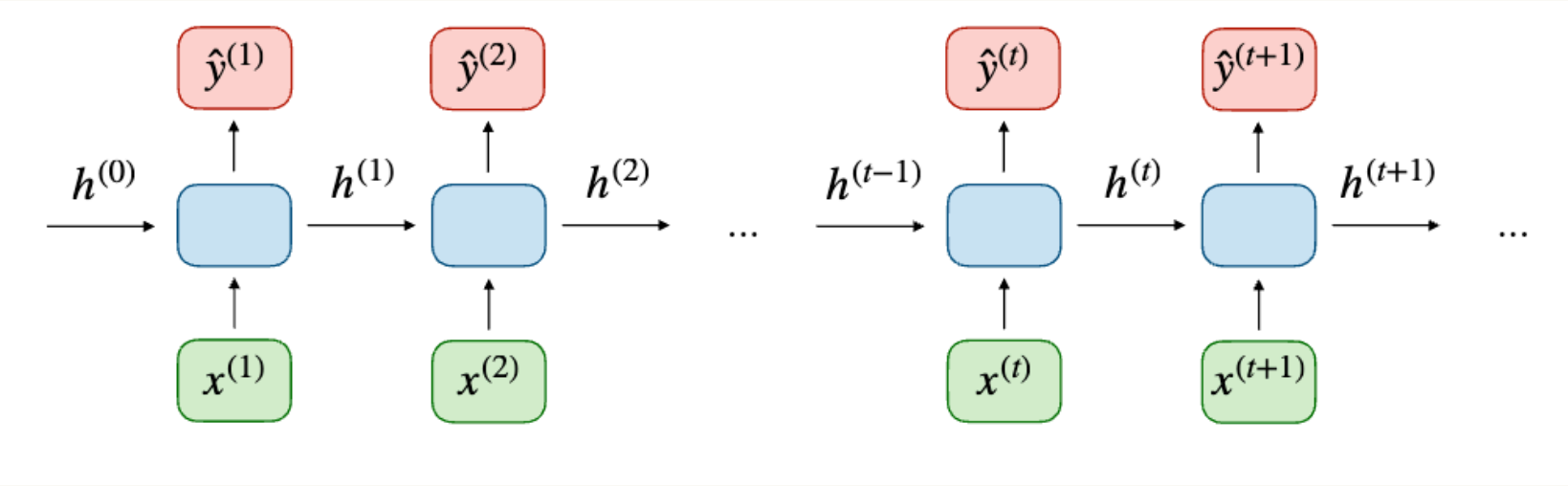
Read in order, keep a running memo

To respect word order, read the document **left to right, one token at a time**, carrying a memo. At each step, update a running summary — the **hidden state** h_t — from the previous summary h_{t-1} and the current token x_t.

> The book's definition: an RNN "keeps a **self-mutating hidden state** to account for temporal inputs" [1, p. 47]. The state is rewritten by every incoming token, and it is the **only channel** through which the past can influence the future.

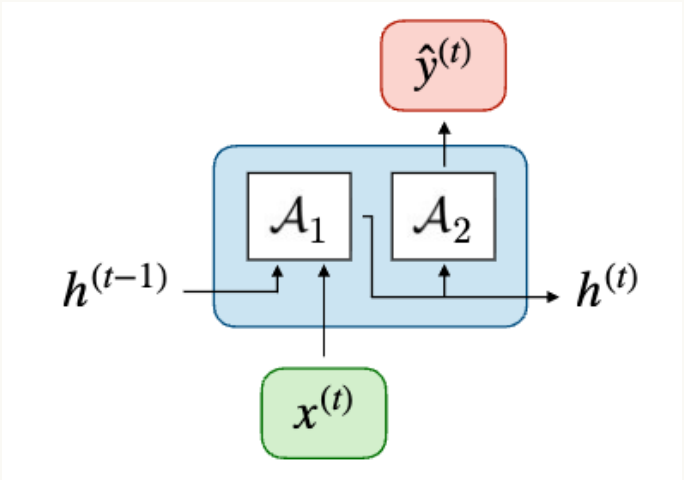
After the last token, h_T is a single vector that has "seen" the whole sequence *in order* — a document embedding that knows `dog bites man` $\neq$ `man bites dog`.

The picture: an RNN unrolled through time



Same block, same weights, applied at every step — shown here both near the start ($h^{(0)} \rightarrow h^{(1)} \rightarrow h^{(2)}$) and at a general step deep in the sequence ($h^{(t-1)} \rightarrow h^{(t)} \rightarrow h^{(t+1)}$): the reader doesn't change between words. After the very last token, that running state becomes the **document embedding**, h_T.

The recurrence, step by step



$$h^{(t)} = g_1\big(W_{hh}\, h^{(t-1)} + W_{hx}\, x^{(t)} + b_h\big)$$
$$\hat{y}^{(t)} = g_2\big(W_{yh}\, h^{(t)} + b_y\big)$$

(6)

- **In the picture:** $\mathcal{A}_1$ reads $h^{(t-1)}$, $x^{(t)}$ and writes the new memory $h^{(t)}$; $\mathcal{A}_2$ reads that same $h^{(t)}$ to predict $\hat{y}^{(t)}$.
- W_{hx} **reads** the token, W_{hh} **carries** the memory, g_1 is usually $\tanh$ — and these weights are **shared across all** t (an RNN is a very deep, *tied*-weight network; this matters next section).
- $h^{(0)}$ is usually the **zero vector** — the blank memo before reading anything.

Worked micro-example: two steps, order A vs. order B

$d = d_h = 2$; $W_{hh} = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \end{pmatrix}$, $W_{hx} = I$, $b_h = 0$, $g_1 = \tanh$, $h_0 = (0, 0)$. Feed the same two token vectors as the mean-pooling example: $x_{\texttt{paper}} = (1, 0)$, $x_{\texttt{lantern}} = (0, 1)$.

Order A — `paper lantern`:

- $t = 1$: pre-activation $= (1, 0) \rightarrow h_1 = (\tanh 1, \tanh 0) = (0.762,\ 0)$.
- $t = 2$: pre-activation $= (0.5 \times 0.762,\ 0 + 1) = (0.381,\ 1) \rightarrow h_2 = (0.364,\ 0.762)$.

Order B — `lantern paper`: same arithmetic, tokens swapped → $h_1 = (0, 0.762)$, $h_2 = (0.762,\ 0.364)$.

> ⚠ $(0.364, 0.762) \neq (0.762, 0.364)$ — while mean-pooling gave $(0.5, 0.5)$ for **both** orders. Order is now *encoded*.

A shadow of what's coming

Look again at Order A: the first token's trace (0.762) reached h_2 only after being **multiplied by W_{hh}** (shrunk to 0.381) and re-squashed by $\tanh$ — and every further step multiplies by W_{hh} again.

> File that observation. The next section turns it into the **vanishing-gradient argument**: the same shrinking that happens forward in the *values* also happens backward in the *gradients*.

Training: loss and backprop through time

For tasks that predict at every step, the loss **sums the per-step losses**:

$$\mathcal{L}(\hat{y}, y) = \sum_{t=1}^{T_y} \mathcal{L}\big(\hat{y}_t,\ y_t\big) \tag{7}$$

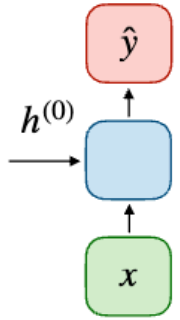
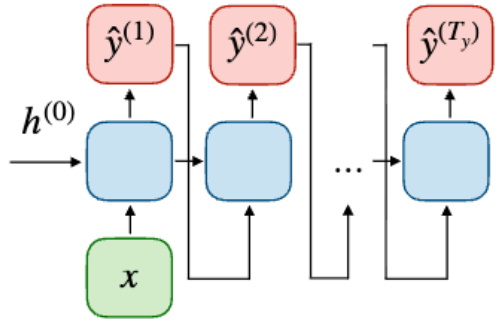
For our many-to-one document encoder, the sum has a **single term** — the loss at the final prediction. Gradients flow **back through time** — the same backprop from Module 0, unrolled along t.

Because the *same* W acts at every step, its gradient **sums the contributions of every unrolled copy**:

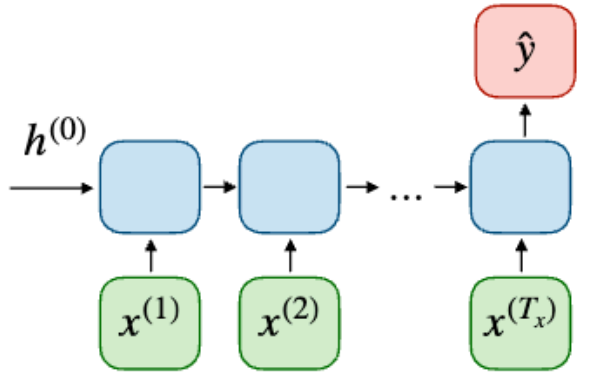
$$\frac{\partial \mathcal{L}^{(T)}}{\partial W} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}^{(T)}}{\partial W}\bigg|_{(t)} \tag{8}$$

Input/output shapes decide the use case

T_x tokens in, T_y predictions out — the shape decides the tool:

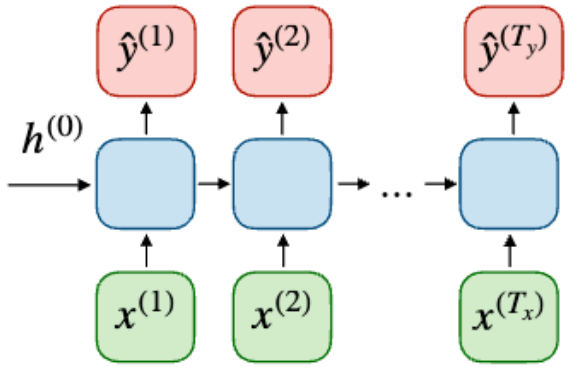
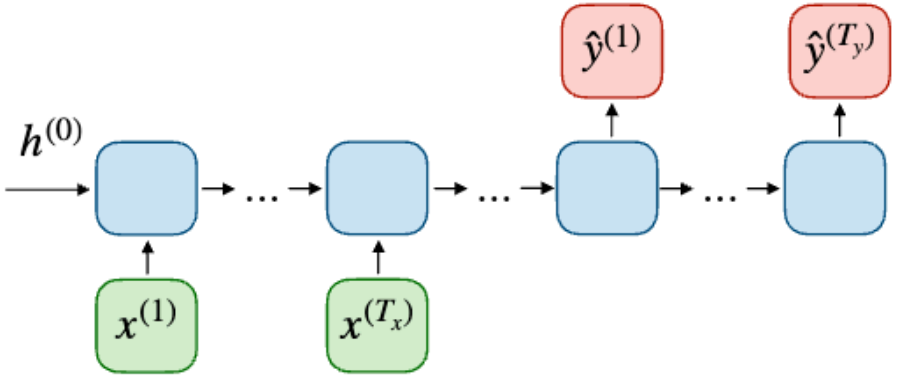
One to one $T_x = 1$ $T_y = 1$	Traditional neural network	
One to many $T_x = 1$ $T_y > 1$	*Use case*: Generation (text, music or other) *Input*: Word *Output*: Rest of the sentence	

Many-to-one — our case

Many to one $T_x > 1$ $T_y = 1$	*Use case*: Sentiment classification *Input*: Sentence *Output*: Sentiment (positive or negative)	

Many tokens in, **one** vector out. The example is sentiment classification; ours is the same shape with a different name: the sentence goes in, and the final state h_T *is* the **document embedding**.

Many-to-many — two flavors

Many to many $T_x = T_y$	*Use case*: Name entity recognition *Input*: Sentence *Output*: Word-level classification	
Many to many $T_x \neq T_y$	*Use case*: Machine translation *Input*: Sentence in source language *Output*: Sentence in target language	

RNN encoders: strengths & limitations

- ✅ Respects word order — the document embedding h_T genuinely differs for different orderings of the same tokens.
- ✅ Parameter count is independent of sequence length (weight tying).
- ⚠️ A vanilla RNN is **left-to-right only** — seeing the future needs a *bidirectional* RNN (two RNNs; we'll meet this again with ELMo).
- ⚠️ The hidden state is a **bottleneck**: everything the model will ever know about the sequence is squeezed through one fixed-size h_T.
- ⚠️ And as the shadow slide hinted: something is off with long sequences. Next.

Part III — The Vanishing-Gradient Problem

The memo gets smudged

To learn that the *first* word matters for a decision made at the *last* word, the gradient has to travel back across **every** step in between — and at each step it gets multiplied by the same matrix W_{hh}.

> ⚠️ Multiply a number smaller than 1 by itself 40 times and it's effectively 0; larger than 1 and it explodes. A plain RNN's memory of the distant past **fades exponentially**.

You already watched this happen *forwards* in the micro-example: `paper`'s trace shrank $0.762 \rightarrow 0.381$ in a single step. The gradient going *backwards* is the same story, repeated for every step it has to cross.

Why it fades: the same factor, over and over

Going back from the loss to an early state h_k, the gradient crosses one step at a time — and **every step contributes the same kind of factor**, because the weights are tied:

$$\frac{\partial \mathcal{L}_T}{\partial h_k} = \frac{\partial \mathcal{L}_T}{\partial h_T} \underbrace{\prod_{t=k+1}^{T} \frac{\partial h_t}{\partial h_{t-1}}}_{T-k \text{ factors, all alike}} \tag{9}$$

Call the typical size of one such factor r. Then crossing a gap of $T - k$ steps means multiplying by r that many times:

$$\left\| \frac{\partial \mathcal{L}_T}{\partial h_k} \right\| \lesssim r^{T-k} \tag{10}$$

$r < 1$ → **vanishes.** $r > 1$ → **explodes.** Only $r \approx 1$ would preserve it — and nothing holds a plain RNN there.

How fast is "exponential"? Put in numbers

r^{T-k} across gaps of 5–40 tokens (gap 40 = the homework's exact padded length $L = 40$):

per-step factor r	gap 5	gap 10	gap 20	gap 40
0.9 (barely shrinking)	0.59	0.35	0.12	0.015
0.45 (realistic)	1.8×10^{-2}	3.4×10^{-4}	1.2×10^{-7}	1.3×10^{-14}
1.1 (barely growing)	1.6	2.6	6.7	45

Even the **optimistic** top row leaves the 40-tokens-ago gradient at ~1.5% of its size. The realistic middle row leaves 10^{-14} — for learning purposes, **zero**.

Part IV — The Gated Cell: LSTM

The fix: a new state, controlled by gates

Add a second state — the **cell state** c_t — a dedicated memory channel, separate from h_t, that carries information across time steps. Small learned **gates** (values in $(0, 1)$) decide, per coordinate: what to *forget* from c_{t-1}, what new content to *add*, and what to *read out* as h_t.

> In the RNN, the **same fixed matrix** W_{hh} multiplies the signal at *every* step — that fixed, uncontrollable factor is what makes the gradient vanish or explode (Part III). The gates replace it with a **dynamic, learned** multiplier: a different value per coordinate and per step, set by the network itself.

A gate, generically

A gate is just a **sigmoid** over the concatenated $[h_{t-1}, x_t]$, squashing to $(0, 1)$ — an elementwise "how much to let through" dial.

$$\Gamma_\bullet = \sigma\big(W_\bullet [h_{t-1}, x_t] + b_\bullet\big), \qquad \bullet \in \{i,\ f,\ o\} \tag{11}$$

Two details that make gates work:

- They are **vectors**, not scalars — each of the d_h memory coordinates gets its own dial, so the cell can hold one feature for 30 steps while overwriting its neighbor every step.
- The squashing must be a **sigmoid**, not $\tanh$: a mask has to be a *fraction* in $(0, 1)$ — a negative "gate" would flip signs, not attenuate.

Three gates, three purposes

Gate	Purpose	Read the extremes
input Γ_i	filter what of the *new* info is useful	$\to 0$: ignore new token · $\to 1$: take it in
forget Γ_f	decide what *old* info to discard	$\to 0$: erase · $\to 1$: keep
output Γ_o	decide what of the state to *expose*	$\to 0$: output nothing · $\to 1$: output everything

Together, Γ_f and Γ_i **interpolate** between holding the old memory and writing the new one — that blend is what makes the cell path additive instead of multiplicative.

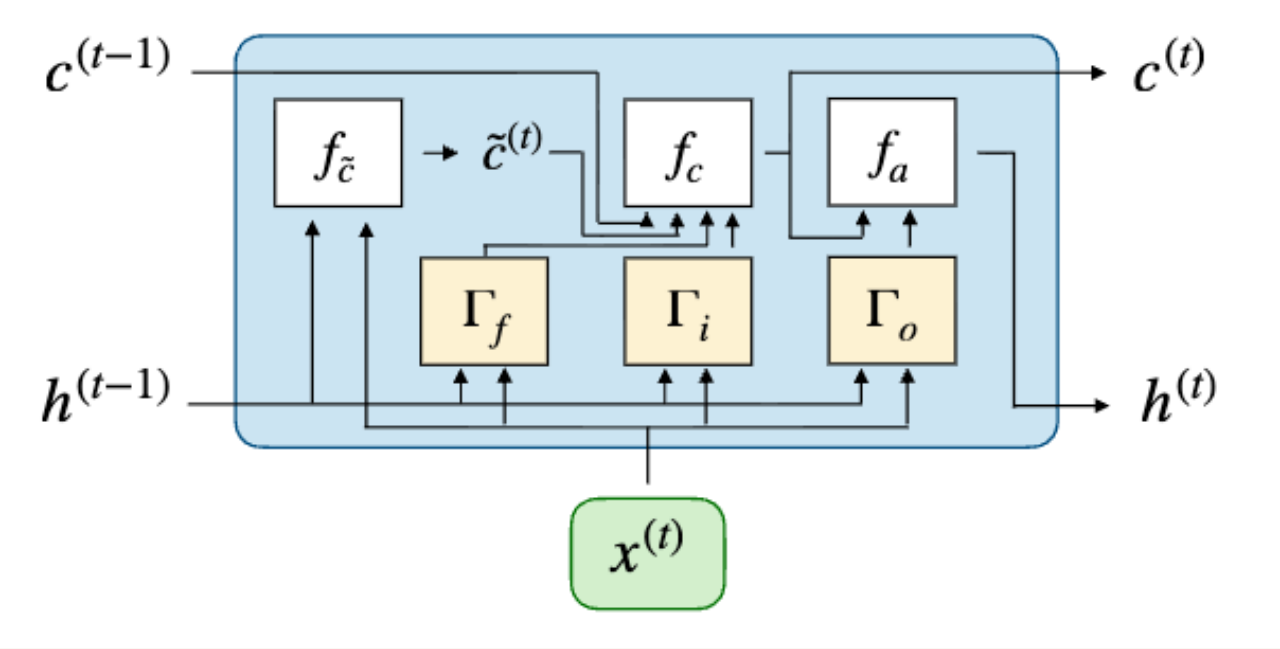
> ⚠️ **Misconception**: "$\Gamma_f = 1$ means forget." Inverted — the forget gate *multiplies* the old cell, so 1 = keep everything, 0 = erase.

LSTM, step by step

$$\tilde{c}_t = \tanh\big(W_c[h_{t-1}, x_t] + b_c\big) \qquad \text{(candidate content)}$$
$$c_t = \Gamma_f \odot c_{t-1} + \Gamma_i \odot \tilde{c}_t \qquad \text{(forget old + add new)}$$
$$h_t = \Gamma_o \odot \tanh(c_t) \qquad \text{(read out)} \tag{12}$$

1. $\tilde{c}_t$ proposes new content ($\tanh$, so it's in $(-1, 1)$).
2. c_t **replaces** the cell state: keep Γ_f of the old, add Γ_i of the new candidate.
3. h_t re-squashes the cell state and **exposes** only Γ_o of it as the visible hidden state.

The picture: the LSTM cell



The cell state $c^{(t-1)} \rightarrow c^{(t)}$ runs along the **top**, steered by the three gates; $f_{\tilde{c}}$ proposes the candidate, f_c writes the new cell state, f_a reads out $h^{(t)}$ — a *gated view* of the cell, not the cell itself.

Why this stops vanishing

Differentiate the cell state:

$$\frac{\partial c_t}{\partial c_{t-1}} = \operatorname{diag}(\Gamma_f) \tag{13}$$

The gradient across time is now $\prod_t \operatorname{diag}(\Gamma_f)$ — a product of **gate values**, not a product of the weight matrix W_{hh}. With $\Gamma_f \approx 1$ this product stays ≈ 1 over many steps: the long-range gradient **survives** (contrast eq. 10).

> The network *learns* when to hold ($\Gamma_f \to 1$) and when to reset ($\Gamma_f \to 0$).

Variants: the formulas aren't unique

The boxed equations are **one** standard formulation, not *the* definition — the book notes the gated formulas "are not unique" [1, pp. 52–53]. Real implementations differ in small ways (peephole connections, coupled input/forget gates, gate ordering), and there are lighter relatives that merge the cell and hidden states into one.

> What every variant shares is the one idea that matters: **an additive path through time, steered by learned gates instead of a repeated matrix multiply.** Get that, and any variant reads as a rearrangement of the same trick.

Remark: ELMo — from static to contextual

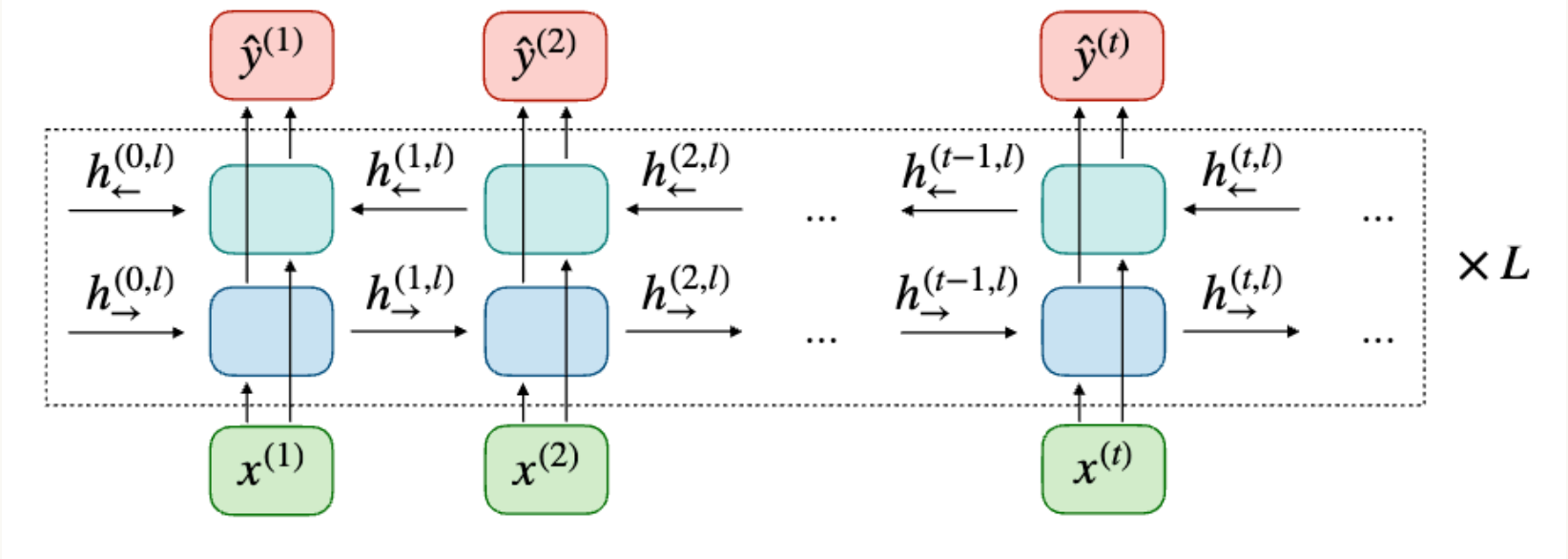
ELMo stacks **bidirectional** LSTMs (one reading →, one reading ←). A token's embedding becomes a function of its **whole sentence** — concretely, the concatenation of both directions' top-layer hidden states:

$$\hat{y}_t = [\overrightarrow{h_{t,L}};\ \overleftarrow{h_{t,L}}] \tag{14}$$

So `bank` gets **different** vectors in "river bank" and "savings bank" — the benefit is a **context-dependent** embedding instead of one fixed vector per token.

> This is the bridge from *static* to *contextual* embeddings; the Transformer (Module 2) takes it further and drops recurrence entirely.

The picture: ELMo



Two independent LSTM stacks read the sentence in opposite directions: $h_{\rightarrow}^{(t,l)}$ (blue, bottom) flows **left→right**, $h_{\leftarrow}^{(t,l)}$ (green, top) flows **right→left** — both stacked $\times L$ layers deep. At each position t, the top layer's two hidden states concatenate into $\hat{y}^{(t)}$ — eq. (14) from the previous slide, now in picture form.

The LSTM: strengths & limitations

- ✅ Additive cell path defeats the *structural* cause of vanishing gradients.
- ✅ Gates are cheap: one linear layer + sigmoid each, sharing the input $[h_{t-1}, x_t]$.
- ⚠️ **"Greatly mitigate," not "eliminate."** Extremely long ranges can still fade — Γ_f only *slows* the decay, it doesn't remove it.
- ⚠️ It is **still sequential**: step t needs step $t-1$'s output, so training and inference can't parallelize across time.
- ⚠️ The document is still funneled through **one** fixed-size final state h_T — a bottleneck. Attention, next, removes it.

Part V — Attention as Weighted Pooling

The memo isn't enough

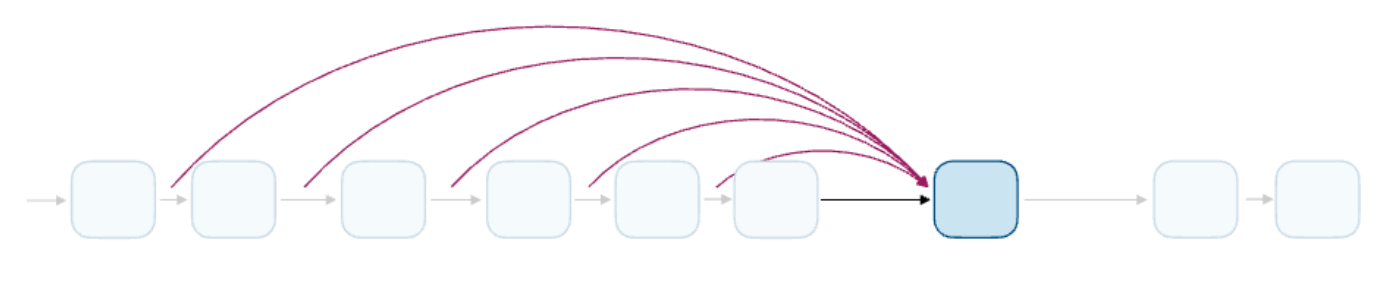
Gates *mitigate* vanishing gradients, but "these methods are imperfect in practice". The stress test is translation: a late output word may need a **specific early input word**, not a blurry summary.

Concretely (our running sentence): `my old paper lantern is glowing` → `mi vieja lámpara de papel está brillando`. To emit `de papel` the decoder must reach back to `paper`; to emit `brillando` it must reach `glowing` — wherever they sit. Squeezing all of that through one fixed-size h_T asks the memo to *be* the document.

Attention removes the bottleneck

An RNN funnels the whole sequence through **one** fixed-size memo h_T. **Attention** instead lets the model **look back at every hidden state at once** and take a **weighted average** — deciding on the fly which past tokens matter most.

The idea, in a picture



Every past hidden state gets a vote (its attention weight, the arc into the current step); the context vector is their weighted sum.

The math, step by step

$$c_i = \sum_{j=1}^{T} \alpha_{i,j}\, h_j, \qquad \alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{j'=1}^{T} \exp(e_{i,j'})}, \qquad e_{i,j} = a(h_{i-1}, h_j) \tag{15}$$

1. c_i = the **context vector** for position i: a convex combination ($\sum_j \alpha_{i,j} = 1$, all ≥ 0) of **all** hidden states.
2. $\alpha_{i,j}$ = attention **weights** — a **softmax** over alignment scores: "how much should position i pay attention to position j."
3. $e_{i,j} = a(h_{i-1}, h_j)$ = the **alignment model** — a small learned scoring function. Bahdanau et al. (2015) [7] use a tiny MLP; Module 2 replaces it with a plain (scaled) dot product.

As a document embedding

With a single learned **query** q, attention *pools* the whole sequence into one vector — the alignment score is a plain **dot product** $q^\top h_j$ between the query and each hidden state:

$$c = \sum_j \alpha_j h_j, \qquad \alpha_j = \mathrm{softmax}_j(q^\top h_j) \tag{16}$$

An order-aware, bottleneck-free summary that usually beats mean-pooling and often the raw h_T.

Why this fixes long-range learning

In an RNN, the loss reaches an early state h_j through $T - j$ Jacobian factors (eq. 9) — the signal decays with distance. Through attention, the loss reaches **every** h_j in **one hop**: $\partial c_i / \partial h_j$ contains the direct term $\alpha_{i,j} I$ — no product over the gap, just a weight.

> **Distance stops mattering to the gradient.** The model *chooses* relevance instead of *inheriting* recency.

Attention pooling: strengths & limitations

- ✅ No bottleneck — every past hidden state is one hop away from the gradient.
- ✅ Recovers mean-pooling as a special case, and can sharpen far beyond it.
- ⚠️ $O(T^2)$ cost — but parallelizable, unlike the RNN's sequential $O(T)$.
- ⚠️ Attention weights show *where* the model looked — suggestive, but not a faithful causal explanation.
- ⚠️ Here, attention **augments** an RNN; in Module 2 it **replaces** it entirely — everything after this lesson builds on eq. (15).

Recap

- **Heuristic pooling** (BOW · TF-IDF · mean-pooling): cheap, surprisingly strong — but **order-blind** (`dog bites man` = `man bites dog`).
- **RNN**: reads in order → order *is* encoded. But weight-tying makes its gradient a **product of $T - k$ Jacobian factors** → vanishes/explodes exponentially with distance.
- **LSTM**: an **additive** cell path steered by *gates*, not matrix multiplies → the gradient is a product of *learned* gate values. Vanishing greatly mitigated.
- **Attention**: every past state is **one hop** from the gradient — no bottleneck, at $O(T^2)$ cost. The equation the rest of the course builds on.

What's next

Lesson 3 turns these vectors into *usable* tools: how do we measure whether two embeddings are "similar" (**cosine similarity**), how do we *see* a high-dimensional cloud of embeddings on a 2-D screen (**PCA**, **t-SNE**), and how do we search millions of them fast without brute-force comparison (**locality-sensitive hashing**)?

Curious now? Bahdanau, Cho & Bengio [7] is worth reading in full for how attention was first introduced — the paper this lesson's eq. (15) comes from, and the direct ancestor of Module 2's Transformer.

References & further reading

- [1] A. Amidi and S. Amidi, *Super Study Guide: Transformers and Large Language Models*, Part 2 "Embeddings," 2024 — primary source for this lesson.
- [2] Y. Bengio, P. Simard, and P. Frasconi, "Learning Long-Term Dependencies with Gradient Descent is Difficult," *IEEE Trans. Neural Networks*, 1994.
- [3] R. Pascanu, T. Mikolov, and Y. Bengio, "On the Difficulty of Training Recurrent Neural Networks," in *Proc. ICML*, 2013 (arXiv:1211.5063).
- [4] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, 1997.
- [5] K. Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," in *Proc. EMNLP*, 2014 (arXiv:1406.1078).
- [6] M. Peters et al., "Deep Contextualized Word Representations" (ELMo), in *Proc. NAACL*, 2018 (arXiv:1802.05365).